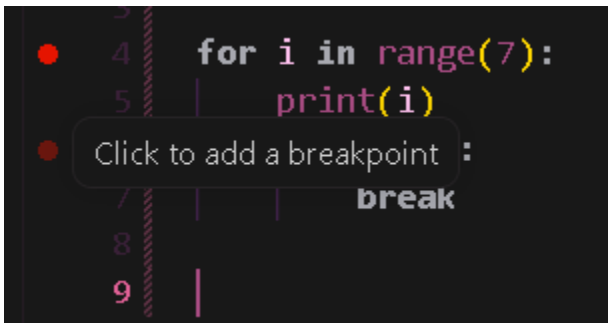
Summer Training Program **WEEK 2 - DAY 6**

- Debugging
- Functions in python
- Basics of recursion

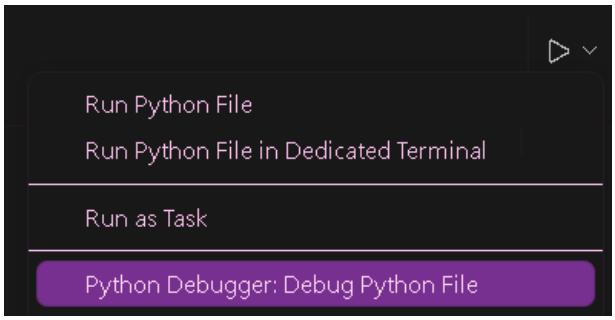
Debugging

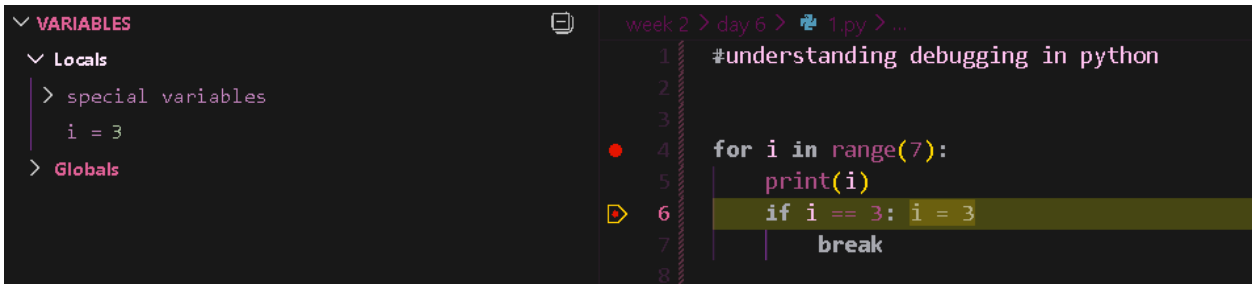
Using VScode:

- We can debug programs in python using vscode
- We add breakpoints on the loop

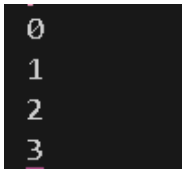


- And then use python debugger





OUTPUT :



Using logs:

- We can simply use print statements in loops to print every iteration.

```
12  for i in range(7):
13      print('i = ', i)
14      if i == 3:
15          print("Breaking the loop at i =", i)
16          break
```

Here statement 13 and 15 are log statements

OUTPUT:

```
i =  0
i =  1
i =  2
i =  3
Breaking the loop at i = 3
```

FUNCTIONS IN PYTHON

A function is a piece of code that contains logic that can be executed again and again

Function definition:

```
5  def main():
6      print("This is a function")
```

- Name of the function can be customized
- Every statement/logic inside the function is written after 4 spaces or 1 tab
- Return is the last statement of a function(if we don't write it, it's written on its own/default) after that no other statement is considered in main.

Function execution:

```
main()

#or

if __name__ == "__main__":
    main()
```

- The function won't execute on its own, we have to write these executing statements to run it.

Taking the example of a function to understand the working better:

- Function to find maximum in a given list of elements:

```
def find_max(numbers):
    max = numbers[0]
    for index in range(1, len(numbers)):
        if numbers[index] > max:
            max = numbers[index]

    print('max in', numbers, 'is:', max)
```

- Suppose we have to use this function on the following lists:

```
data = [10, 20, 50, 70, 30, 15]
scores = [27, 110, 35, 89, 20, 30]
prices = [1500, 3000, 5000, 1200, 4500]
```

- There are multiple ways to execute these functions:
  We can use either
  1. Reference copy operation:

     ```
     find_max(numbers=data)
     ```

     OUTPUT:

     ```
     max in [10, 20, 50, 70, 30, 15] is: 70
     ```

  2. Simple/ informal way:

     ```
     find_max(scores)
     ```

     OUTPUT:

     ```
     max in [27, 110, 35, 89, 20, 30] is: 110
     ```

  3. Inputting the data itself:

     ```
     find_max([1500, 3000, 5000, 1200, 4500])
     ```

     OUTPUT:

     ```
     max in [1500, 3000, 5000, 1200, 4500] is: 5000
     ```

Understanding parameters and arguments:

- A function can have many arguments.
- These can even be assigned a default value.
- But the default values have to be strictly assigned to the right most parameters first.

```python
#understanding parameters and arguments in python

def add_numbers(a, b):
    print('a =', a)
    print('b =', b)
    print('sum =', a + b)

def multiply_numbers(x, y=1):
    print('x =', x)
    print('y =', y)
    print('product =', x * y)

add_numbers(10, 20)
multiply_numbers(10)
```

OUTPUT:

```
a = 10
b = 20
sum = 30
x = 10
y = 1
product = 10
```

def add_numbers(number1=10, number2): # GIVES AN ERROR

Reference copy operation on functions:

- Reference copy operation can also be used on functions in python

```python
def add_numbers(a, b):
    print('a =', a)
    print('b =', b)
    print('sum =', a + b)

add=add_numbers #refrence copy operation

add(10, 20) #calling the function using refrence copy

add_numbers(30, 40) #calling the function using original name
```

OUTPUT:

```
a = 10
b = 20
sum = 30
a = 30
b = 40
sum = 70
```

Function deletion:

- Functions can be easily deleted in python using the “del” keyword.
- Note: all the reference copies of the function will also get deleted.

del(add_numbers) #deletes the function add_numbers

Function redefination:

- Functions in python can be refined.

```
#function re defination in python

def add_numbers(a,b):
    return a+b

def add_numbers(a,b,c):      #the last copy of function will be used in python
    return a+b+c


print('sum is', add_numbers(10,20,30)) #calling the function using 3 parameters
```

OUTPUT:

```
sum is 60
```

- The latest copy of the function is executed in case of redefination
- Hence in this case #add_numbers(10,20) will give an error because the latest function has 3 arguments
- The old function is destructed and the new one is assigned in the memory
- If we don't wish to the older function, we can use the reference copy operation to assign the older function a new name say, # add=add_numbers
- After that we can perform redefination as it is.

```
def add_numbers(a,b):
    return a+b

add=add_numbers #refrence copy operation

def add_numbers(a,b,c):
    return a+b+c

print('sum is', add(10,20))
print('sum is', add_numbers(10,20,30))
```

OUTPUT:

```
sum is 30
sum is 60
```

Variable arguments/ star arguments in python [extremely important for interviews]

- Using these arguments in python, we can pass a variable amount of parameters
- Syntax: '*args' is passed as the argument to the function
- It makes a tuple of all the elements passed to the function

```
1   # Variable Arguments in Python
2   def add_numbers(*args):
3       print(args)
4       print(type(args))
5       print(sum(args))
6
7
8   add_numbers(10, 20, 30, 40, 50)
9   add_numbers(10, 20, 30)
10  add_numbers(-10, -20, -30)
```

OUTPUT:

```
(10, 20, 30, 40, 50)
<class 'tuple'>
150
(10, 20, 30)
<class 'tuple'>
60
(-10, -20, -30)
<class 'tuple'>
-60
```

- We can even pass heterogeneous parameters:
  #add_numbers(10,20,30, 'john')
- A tuple is used because it is an immutable data structure, it saves computation, time and memory.

Keyword variable arguments/ star star arguments [extremely important for interviews]

- These also work similar to the star arguments, but in this case, it makes a dictionary of all the values passed to the function.
- But it is our choice, we can pass both tuple type and dictionary type parameters.

```
# Keyword Arguments -> KeyWord Variable Arguments
def fun(**kwargs):
    print(kwargs)
    print(type(kwargs))


fun(a=10, b=20, c=30, d='john')
fun(name='john', email='john@example.com', phone='+919876543210')
```

OUTPUT:

```
{'a': 10, 'b': 20, 'c': 30, 'd': 'john'}
<class 'dict'>
{'name': 'john', 'email': 'john@example.com', 'phone': '+919876543210'}
<class 'dict'>
```

Using args and kwargs together:

- We can pass both tuple and dictionary type parameters together also.

```
def fun(*args, **kwargs):
    print(args)
    print(kwargs)

fun(10, 'john', 30, a=10, b=20, c=30)
```

OUTPUT:

```
(10, 'john', 30)
{'a': 10, 'b': 20, 'c': 30}
```

Some important facts:

- Once the return statement of the function is executed, the entire frame of that function is deleted/popped from the stack in memory (same works with main function).
- Hence the process dies, and RAM is freed.

An important concept:

When we pass a data value to the function, we use reference copy operation, so if the data is modified in the function, its original value will also get modified

- Scenario:

```
def square_of_numbers(numbers):
    print('[square_of_number] start')
    print('[square_of_number] Before', numbers, id(numbers))

    for index in range(len(numbers)):
        numbers[index] = numbers[index] * numbers[index]

    print('[square_of_number] After', numbers, id(numbers))
    print('[square_of_number] end')
    # return


data=[10, 20, 30, 40, 50]
square_of_numbers(numbers=data)

print('After calling the function', data, id(data))
```

- Here data is passed into the function and after function execution, the value of data is also changed because both numbers and data point to the same container in the memory.
- OUTPUT:

```
[square_of_number] start
[square_of_number] Before [10, 20, 30, 40, 50] 1986479087744
[square_of_number] After [100, 400, 900, 1600, 2500] 1986479087744
[square_of_number] end
After calling the function [100, 400, 900, 1600, 2500] 1986479087744
```

How to fix this situation so that the original data does not change?

- We have to use a new variable inside the function that points to a different memory location.

```
    my_numbers = []
    for index in range(len(numbers)):
        my_numbers.append(numbers[index])
```

- We make #my_numbers as an empty list and copy the values of numbers into it.
- Here we can not use the reference copy operation, since it will point to the same value only.
- So we use the append function for this purpose

Corrected code:

```
def square_of_numbers(numbers):
    print('[square_of_number] start')
    print('[square_of_number] Before', numbers, id(numbers))

    #using a new variable that points to a seperate memory location
    my_numbers = []
    for index in range(len(numbers)):
        my_numbers.append(numbers[index])

    print('[square_of_number] Before', my_numbers, id(my_numbers))

    for index in range(len(my_numbers)):
        my_numbers[index] = my_numbers[index] * my_numbers[index]

    print('[square_of_number] After', my_numbers, id(my_numbers))
    print('[square_of_number] After', numbers, id(numbers))
    print('[square_of_number] end')
    # return
```

OUTPUT:

```
[square_of_number] start
[square_of_number] Before [10, 20, 30, 40, 50] 2076027609216
[square_of_number] Before [10, 20, 30, 40, 50] 2076027757888
[square_of_number] After [100, 400, 900, 1600, 2500] 2076027757888
[square_of_number] After [10, 20, 30, 40, 50] 2076027609216
[square_of_number] end
After calling the function [10, 20, 30, 40, 50] 2076027609216
```

This method is called deep copy. [both shallow copy and deep copy have now been discussed]

Recursion:

When a function executes itself from the same function, the process is called recursion.

Eg:

```
def print_number(number):
    if number <=10:
        print(number)
        print_number(number+1) # execute the same function again from itself

    # return


print_number(5)
```

OUTPUT:

```
5
6
7
8
9
10
```

ASSIGNMENT:

- **Perform the deep copy operation**
- **Use breakpoints to debug code in VScode**

  (PUSHED TO REPO)

GURASSIS KAUR
g.assiskaur@gmail.com